# VMI Analysis GUI

User Manual

Velocity-map imaging photoelectron analysis
and the NiceGUI web interface

| | |
|---|---|
| Application version | 0.1.0 |
| Document revision | 1.0 |
| Date | November 10, 2026 |
| License | MIT |

This manual describes the panels, input fields, processing stages, output files, and operating constraints of the application.

# Contents

**1 Introduction** **3**
1.1 The analysis chain . . . 3
1.2 How this manual is organised . . . 3

**2 Installation and first run** **4**
2.1 Requirements . . . 4
2.2 Install . . . 4
2.3 Start the interface . . . 4
2.4 A first end-to-end run . . . 4
2.5 What to check after the first run . . . 4

**3 The shared interface** **5**
3.1 Window layout and tabs . . . 5
3.2 The Run/Stop control . . . 5
3.3 Job queue, concurrency and file reservations . . . 5
3.4 The log console and progress . . . 5
3.5 The results area . . . 6
3.6 The local file picker . . . 6
3.7 The image inspection overlay . . . 6
3.8 Saved settings . . . 6
3.9 Server lifecycle, ports and reconnecting . . . 7

**4 Data you must provide** **7**
4.1 Camera files (`RAW-*.h5`) . . . 7
4.2 GMD timestamp file (`current-time-both.txt`) . . . 7
4.3 Dark frames . . . 7
4.4 Circularization calibration files . . . 7
4.5 Workspace and output folders . . . 8

**5 Panel reference** **8**
5.1 GMD Map . . . 8
5.2 Movie . . . 9
5.3 Dark Frame . . . 10
5.4 Accumulate . . . 11
5.4.1 The frame histogram preview . . . 12
5.5 Centroiding . . . 12
5.6 Binning . . . 17
5.7 Pipeline . . . 18
5.8 Circularize . . . 19
5.9 Trig Calibration . . . 21
5.10 PES / Abel inversion . . . 23

**6 Output files** **24**

**7 Energy calibration** **24**
7.1 The Xe 4d reference . . . 24
7.2 Procedure . . . 25
7.3 Working with the spectrum file . . . 25

**8 Command-line reference** **25**
8.1 Conventions shared by all programs . . . 26
8.2 Environment variables . . . 26

**9 Troubleshooting and constraints** **26**

**A Glossary** **27**

**B Formulas used by the pipeline** **27**

# 1 Introduction

This application turns raw camera frames from a velocity-map imaging (VMI) photoelectron experiment into calibrated photoelectron energy spectra and angular-anisotropy ($\beta$) parameters. The repository contains two halves that are shipped together:

- a set of standalone Python command-line programs that perform the numerical analysis, and
- a NiceGUI web interface with one panel per program plus a pipeline panel that runs the whole chain in order.

No experiment data is bundled with the repository. Every path used by the application is supplied by the user through the interface or on the command line.

## 1.1 The analysis chain

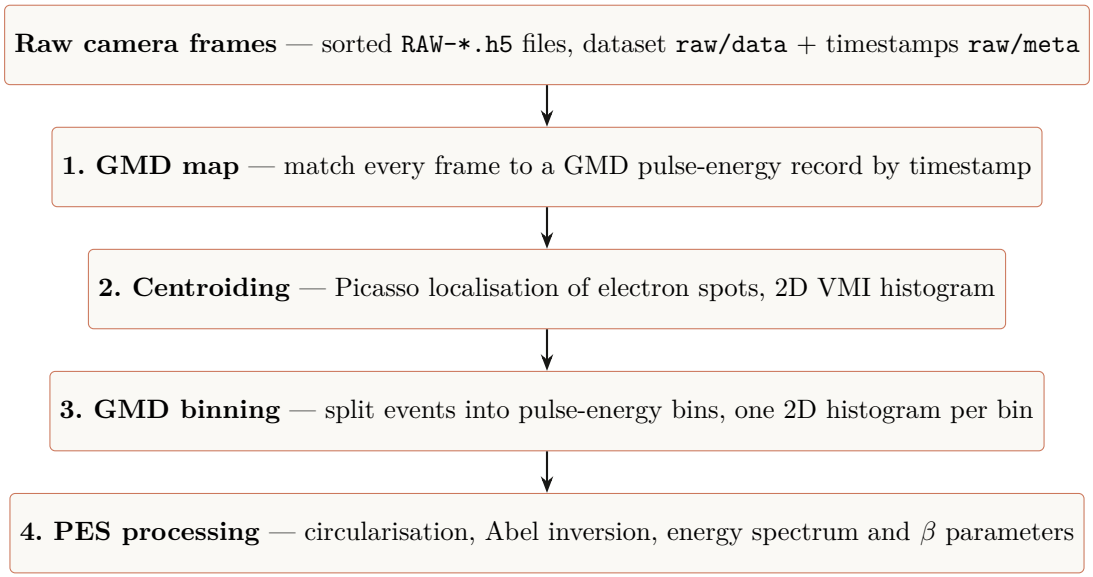



Figure 1: The four-stage analysis pipeline. Stages 1–4 are available as individual panels and as a single Pipeline run.

Every stage is also a standalone command-line program, so the interface is optional: a user can run the same steps from a terminal. The panels launch those programs as subprocesses; they never import analysis code into the web process. This keeps a crashed or cancelled job from taking down the interface and makes long runs cancellable.

## 1.2 How this manual is organised

Section 2 explains installation, starting the interface, and a first end-to-end run. Section 3 describes behaviour shared by every panel: the window layout, the Run/Stop control, the job queue, the log console, the file picker, the image-inspection overlay, and saved settings. Then Section 4 describes the files the user must supply. Section 5 is the reference: one subsection per panel, with every field and button. Sections 6 and 7 explain the files produced and the energy calibration procedure, and Sections 8–9 cover command-line use and troubleshooting. The appendices hold a glossary and the main formulas.

Conventions used in the field tables: *verbatim* labels reproduce the interface text exactly; “saved key” is the name under which the value is persisted (Section 3.8); “CLI flag” is the equivalent command-line argument; “default” is the value shown on a fresh installation, i.e. before any saved setting exists.

# 2 Installation and first run

## 2.1 Requirements

- Python 3.12 or newer and `uv`, the project manager used for installation and for launching the application.
- Optional: `ffmpeg` on the system path, needed only to write MP4 movies in the Movie panel (otherwise a GIF is written).
- Optional: MATLAB, needed only for the CPBASEX inversion backend in PES. The default PyAbel backend is pure Python.

## 2.2 Install

```
git clone <repository-url>
cd vmi-analysis-gui
uv sync
```

`uv sync` creates the virtual environment and installs every dependency, including the web interface. The package is installed in editable mode, so later source edits take effect without reinstalling.

## 2.3 Start the interface

```
uv run vmi-analysis
```

The application serves `http://localhost:3157` and opens a browser. Set `VMI_GUI_PORT` to use another port and `VMI_GUI_NO_SHOW=1` to suppress the browser window. If the port is busy, the program prints a suggested free port with ready-to-paste commands and exits; it does not switch automatically.

Each analysis stage can also be run without the interface: `uv run python -m vmi_analysis.<module> --help` lists the options of every program (Section 8).

## 2.4 A first end-to-end run

1. **GMD Map.** Choose the folder of `RAW-*.h5` files and the GMD timestamp file, choose an output map path, and Run. The result is a frame-to-GMD map.
2. **Centroiding.** Choose the same HDF5 folder and an output directory. First tick *Tuning mode only*, set a tuning frame, and Run to inspect the detection; adjust box size or minimum net gradient if spots are missed or noise dominates. Then untick it and run the full batch. Expect a checkpoint, a 2D histogram, and summary figures.
3. **Binning.** Choose the checkpoint, the GMD map, the output directory, the threshold values, and the weighting mode. Run to produce one 2D histogram per GMD bin.
4. **PES.** Put the two circularisation calibration files in `circularization/`, keep the PyAbel backend, and Run. The results are energy spectra, anisotropy parameters, and an archive of all arrays.

Alternatively, after preparing the GMD timestamp file, open the Pipeline tab, fill in the dataset, the GMD file and the map output, and press `Run pipeline`. It runs stages 1–4 in order and skips stages whose outputs already exist.

## 2.5 What to check after the first run

- The GMD map has two header lines plus one row per frame; the console reports the time-difference statistics and the share of frames within 20 and 40 ms.
- The Centroiding tuning figure shows localisations on real spots, and the final summary shows a VMI image with a physically plausible spectrum.

- Binning produced the expected number of `.npy` files (thresholds plus one) with the expected suffix for the chosen weight mode.
- PES produced the three overview figures and the `.npz`; the energy axis is in pixel$^2$ until an alpha is supplied (Section 7).

# 3 The shared interface

All panels follow the same pattern: a section heading with a one-line description, a cream feature card holding the parameter form, one Run/Stop control, a dark log console, and a results area below it. This section describes the parts that behave identically everywhere; the panel-specific fields are in Section 5.

## 3.1 Window layout and tabs

The header shows the title *VMI Analysis*, the subtitle `photoelectron imaging toolbox`, and the *Maximum concurrent jobs* selector. Below it is one row of tabs, in this order: `GMD Map`, *Movie*, *Dark Frame*, *Accumulate*, `Centroiding`, *Binning*, *Circularize*, *PES*, `Pipeline`. The footer reports that independent jobs run concurrently and that closing the browser tab stops the server.

Every panel is built when the page loads and keeps its own run state, progress, log, and results. Switching tabs does not stop or restart work.

## 3.2 The Run/Stop control

Each panel has exactly one action button. It reads *Run* when the panel is idle and changes to:

- *Stop* while the panel's job is running,
- *Cancel queued* while it is waiting for a slot or for a file conflict to clear,
- a disabled *Stopping…* during cleanup.

Stopping a queued job removes it from the queue immediately. Stopping a running job terminates the whole subprocess tree (on Windows with `taskkill /T /F`), so the analysis program and its worker processes end together. Stopping one panel's job never stops another panel's job. The button never starts a second job from a stale click, and it returns to `Run` when the job finishes, fails, or is cancelled.

A job that is active when the page is reloaded is reattached by its panel from the job name, restoring its Run/Stop state, progress, and current log. A job that had already finished before the reload is not restored.

## 3.3 Job queue, concurrency and file reservations

Jobs from all panels share one FIFO queue. The header selector `Maximum concurrent jobs` sets how many may run at once; the default is 2 and the dropdown offers 1–9. Lowering the limit never kills running jobs: they finish, and new jobs start more slowly. The value is saved and survives a restart.

The queue also enforces file safety. Each submitted command declares the files and directories it reads and writes. A job whose output overlaps another job's input or output waits, and shows a reason such as `Waiting for <job>: <path>` or *Waiting for a concurrent job slot.* Unrelated jobs can start in the free slots. Only one job per panel may be active at a time. Jobs launched by the Pipeline panel reserve the inputs and outputs of all four stages for the whole run.

The MATLAB/CPBASEX backend takes an exclusive lock because the MATLAB script uses a shared working file, so MATLAB jobs never overlap each other or a job writing the same directory.

## 3.4 The log console and progress

Every panel has a dark console under the form. It shows the exact command that was launched, prefixed with `[gui] $`, followed by the merged standard output and standard error of the program.

Output is decoded as UTF-8 and streamed while the job runs; the console keeps the most recent 2000 lines, and the panel refreshes it about every 0.7 seconds.

Lines in the standard tqdm form `NN% |...` drive the progress bar. Some stages (for example Circularize and PES) do not print that form, so their bar stays empty while the spinner indicates activity.

### 3.5 The results area

Below the console, a results area is built once when the job reaches a terminal state (*done*, *failed*, or *cancelled*). Most panels show results only for a successful job; a failed job leaves the area empty and the console contains the traceback. Results include:

- interactive Plotly figures (hover for values, drag to zoom, choose *Pan*, double-click or *Reset axes* to restore),
- images and videos served from local paths, with a click-to-inspect overlay (Section 3.7),
- *open containing folder* buttons, and
- the exact output paths and a summary of what was written.

A few panels also offer *View saved result(s)* and `Clear output(s)` buttons. The first re-renders existing outputs without rerunning (it refuses while a job is active); the second asks for confirmation and deletes the listed outputs. These buttons are not present on every panel; the panel reference states which ones have them.

### 3.6 The local file picker

Every path field has a round folder button that opens a server-side file picker. The application and the data are on the same machine, so nothing is uploaded; the dialog browses the local disks directly.

The dialog title matches the field: *Select file*, *Select folder*, or *Save as*. It has an up-arrow button, the current path, and a *Drive* dropdown listing the available Windows drives. Folders are listed before files; dot-files are hidden; file and save modes apply the field's extension filter (for example `.txt`, `.npy`, `.h5`), while folder mode lists folders only. Long directories stop after 1000 entries with a note that more are not shown.

`New folder` creates a folder in the directory currently shown, then opens it. Names containing path separators, Windows-reserved characters, or reserved device names (`CON`, `NUL`, `COM1`, ...) are rejected, and an existing entry is never overwritten.

### 3.7 The image inspection overlay

Clicking any result image opens a dark full-screen overlay with the image and a toolbar: *Pan*, *Crop area*, *+*, *−*, *reset*, and ×. In *Pan* mode, drag to move and use the mouse wheel to zoom around the pointer. *Crop area* lets you drag a rectangle; on release the overlay keeps only that region, enlarged to fit. Repeated crops operate on the current preview, and *reset* restores the whole image. Escape cancels an active selection, or closes the overlay when no selection is active. Zooming and cropping are display-only: they never change the saved image or the analysis data.

### 3.8 Saved settings

Form values are stored server-side in `.nicegui/storage-general.json` inside the workspace. Editing a field saves it immediately, even without clicking Run; the values survive a server restart and are shared by every browser connected to the same server. Job submissions take a snapshot of the inputs, so editing a form after starting a job does not change that job or where its results appear.

Saved keys follow the pattern `<panel>.<field>` and are listed in the field tables of Section 5. Some panels derive initial values from another panel's saved settings (for example PES suggests the saved Centroiding output directory). Legacy `centroiding/config.toml` values are migrated once, read-only, for Centroiding and Binning; no GUI job passes a configuration file to a script.

### 3.9 Server lifecycle, ports and reconnecting

Launch the application with `uv run vmi-analysis`; it serves `http://localhost:3157` and opens a browser. The environment variable `VMI_GUI_PORT` selects another port, and `VMI_GUI_NO_SHOW=1` suppresses the automatic browser window. If the port is unavailable the program prints an error plus a ready-to-paste suggestion for a free port; it does not switch ports on its own.

When the last browser client disconnects, the server waits a one-second reconnect grace period, checks once more, and then shuts down: queued jobs are cancelled, running process trees are terminated, and cleanup is awaited. A fast reload reconnects within the grace period; a slower reload may require starting the server again. Multiple tabs keep it alive, and `Ctrl+C` in the terminal also shuts it down.

## 4 Data you must provide

The repository contains **no experiment data**. Analysis paths are always supplied by the user through the interface or on the command line. This section describes the files the pipeline expects.

### 4.1 Camera files (`RAW-*.h5`)

A dataset is a folder of HDF5 files, usually named `RAW-*.h5`. Each file holds two datasets:

| Dataset | Shape | Content |
|---|---|---|
| *raw/data* | `(N, 1024, 1024) uint16` | Camera image frames. |
| *raw/meta* | `(N, 5) uint64` | Per-frame metadata, five columns. |

The metadata columns are: 0 = frame counter, 1 = high-resolution hardware timestamp, 2 = status flag, 3 = Unix seconds, 4 = nanosecond offset, so the frame timestamp is `col3 + col4/1e9`. The camera and the free-electron laser both run at about 25 Hz. Some frames carry zeroed metadata (`col3 = col4 = 0`); their timestamps are meaningless and should be ignored when judging GMD matching quality.

The number of frames per file is typically about 126, but the pipeline reads `N` from the file, so other counts also work. Individual tools differ slightly in which files they scan (for example `RAW-*.h5` versus all `*.h5`); the panel reference notes this where it matters.

### 4.2 GMD timestamp file (`current-time-both.txt`)

GMD (gas monitor detector) binning and weighting need a text table of pulse energies with timestamps. The file has four whitespace-separated columns: index, timestamp, GMD value 1, GMD value 2. The GMD Map stage matches camera frames to the **second** column (0-based index 1) and copies both GMD values into its map.

### 4.3 Dark frames

A dark frame is a `.npy` array (usually `float64`) of the same shape as one camera frame. The Dark Frame panel computes one from a folder of dark acquisitions; the Centroiding, Movie, and Accumulate panels accept a dark file and subtract it before processing. If no dark file is given, the raw pixel values are used.

### 4.4 Circularization calibration files

The PES stage requires two per-experiment text files in `circularization/`:

`extracted-FOC100eV-20260710.txt` A Gascooke circularization software export: one block per fitted ring with a Gaussian centre (radius in pixels) and the trigonometric terms $\sin(n\theta)$, $\cos(n\theta)$ and their 1-$\sigma$ uncertainties.

`reassignment-FOC100eV-20260710.txt` A two-column radial reassignment map `r_circ r_fixed`. For the reference dataset this was the identity map, so only the azimuthal terms act and any radial nonlinearity propagates into the energy axis.

These files are per-experiment inputs. The extracted trig-fit file can be produced by the Trig Calibration panel (Section 5.9) or copied from the Gascooke software; the reassignment file is supplied separately (an identity map is valid). The Circularize panel takes their paths as fields (and pre-fills the known names); PES looks for exactly these names and warns when either is missing.

### 4.5 Workspace and output folders

In a checkout the workspace is the repository root; an installed copy uses the launch directory unless `VMI_WORKSPACE_DIR` is set. The following locations are conventional:

| Path | Created by | Purpose |
|---|---|---|
| *centroiding/* | user / GUI | Centroiding working data and default outputs |
| *circularization/* | user / GUI | Calibration files and PES results |
| *.cache/vmi-analysis/* | the GUI | Disposable previews, frame dumps, screenshots |
| *.nicegui/* | the GUI | Saved form settings |
| *vendor/pbasex-matlab/* | repository | Bundled MATLAB CPBASEX code |

Analysis outputs go to the directories the user selects; the defaults shown in the panels already point at the conventional folders. Nothing outside the selected output directories is modified by the analysis scripts.

## 5 Panel reference

This section documents every panel in tab order. Each subsection gives the panel's purpose and the command-line module it launches, a table of every input field in user-interface order, the buttons and controls, what the results and log show, the output files, and panel-specific pitfalls.

Field tables use these columns: *Field* (the verbatim label), `Type and default` (widget kind and the value on a fresh installation), `Saved key` (the persistent setting name; "–" means the value is not saved), *CLI flag* (the equivalent command-line argument), and `Meaning`. Numeric values are raw counts, pixels, or seconds where stated. A cleared number field is usually sent as an empty value; where that fails, the table says so.

### 5.1 GMD Map

**Purpose.** Match every camera frame to its nearest GMD pulse-energy record by timestamp. The resulting map is required before GMD weighting (Accumulate) or GMD binning (Binning). The panel launches `vmi_analysis.gmd.build_gmd_map`.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *HDF5 directory* | folder, empty | `gmd_map.h5_dir` | *--h5-dir* | Folder scanned for `RAW-*.h5` files, sorted by name. Required. |
| `GMD timestamp file (current-time-both.txt)` | file, empty | `gmd_map.gmd_path` | `--gmd-path` | The GMD table (Section 4.2); column 2 is the timestamp, columns 3–4 are copied as GMD values. Required. |
| *Output map file* | save, `gmd_map_all.txt` in the workspace | `gmd_map.output_path` | `--output-path` | Destination map. It is overwritten without asking; its folder must already exist. |
| *First N files (0 = all)* | number, 0 | `gmd_map.n_files` | *--n-files* | Analyse only the first $N$ sorted files. 0 means all. A map built for a subset is valid only for that subset. |

**Buttons.** *Run/Stop* submits the four values as job `GMD map`. *Clear output* asks for confirmation and deletes the output file so the next run rewrites it. *open containing folder* appears in the results and opens the output's folder in the file manager.

**Results and log.** On success the results show `Map written: <path> (<n> lines, <size> MB)`; the line count includes two `#` header lines. The map is a whitespace-separated text file with two header lines and one row per frame:

```
frame_idx h5_file frame_in_file gmd_idx gmd_val1 gmd_val2 time_diff_ms
```

Indices are 0-based and `time_diff_ms` is signed. The console prints the number of GMD entries, the frame count, matching progress, time-difference statistics (mean, standard deviation, minimum, maximum and the fraction of frames within 20 ms and 40 ms), and a sequential check of the frame-to-GMD step.

**Pitfalls.** There is no resume: an existing map is overwritten silently. Matching is nearest-neighbour; exact ties pick the earlier GMD entry. Frames with zeroed metadata get timestamp 0 and match the first GMD entry, which pollutes the time-difference statistics but does not stop the run. An empty HDF5 folder produces a header-only map and still reports success, so check the line count. Clear output uses the current field value, not the last submitted path.

## 5.2 Movie

**Purpose.** Render the first $N$ frames as a movie and inspect them frame by frame, for quality control of spot structure, background and detector artifacts. The panel launches `vmi_analysis.imaging.h5_play_frames_movie`; in the GUI it always runs headless, and with *Save movie file* unchecked it plays live in the page instead of writing a file.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *HDF5 directory* | folder, empty | `movie.h5_dir` | *--h5-dir* | Folder of HDF5 files; all `*.h5` files are used, sorted by name, frames in file order. Required. The folder name becomes the movie file name. |
| *Frame dataset* | text, `raw/data` | `movie.h5_dataset` | *--h5-dataset* | Dataset path inside each HDF5 file. |
| `Dark frame .npy (optional)` | file, empty | `movie.dark_path` | *--dark-path* | Subtracted from every frame before display; the result is clipped at 0 so a log scale is defined. Shape is not checked here. |
| *Frames* | number, 20 | `movie.n_frames` | `--n-frames` | Total frames across files. 0 lists no frames and produces no movie. |
| *Interval (ms)* | number, 500 | `movie.interval_ms` | *--interval-ms* | Playback delay, stream-dump pace, and (when the saved fps is 0) the saved frame rate. The in-page player clamps it to at least 40 ms. |
| *Scale frames* | number, 10 | `movie.scale_frames` | *--scale-frames* | The first $k$ frames define one fixed colour maximum for the whole movie, so brightness is comparable across frames. 0 breaks the estimate. |
| *Colormap* | text, `gray` | `movie.cmap` | `--cmap` | Matplotlib colormap name for the frame images. |
| *Log scale* | checkbox, off | `movie.log_scale` | *--log-scale* | Logarithmic colour normalisation instead of linear. |
| *vmin fraction* | number, 0.5 | `movie.vmin_frac` | *--vmin-frac* | Lower colour limit as a fraction of the maximum. |
| *vmax percentile* | number, 99.9 | `movie.vmax_pct` | *--vmax-pct* | Percentile of the first *Scale frames* used as the colour maximum. |
| *Save movie file* | checkbox, checked | `movie.save_movie` | *--save-movie* / `--no-save-movie` | Unchecked writes no movie and enables live playback in the page. It also enables or disables the next three fields. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Movie output folder* | folder, workspace `movies/` | `movie.movie_dir` | `--movie-dir` | Where the movie is written; created on demand. Disabled when not saving. |
| *Format* | select, `.mp4` | `movie.movie_ext` | *--movie-ext* | MP4 needs ffmpeg; the program writes a GIF instead when ffmpeg is unavailable. Disabled when not saving. |
| *Saved fps (0 = playback)* | number, 0 | `movie.movie_fps` | *--movie-fps* | 0 saves at the playback pace. Disabled when not saving. |

**Buttons and controls.** *Run/Stop* submits job `Frame movie`. *Clear outputs* lists and deletes the saved movie of the current dataset name and all dumped frame images. With saving disabled, `Pause`/*Resume* freezes the in-page playback (the job itself is not paused), and the results include a frame slider. A folder button opens the movie's containing folder.

**Results and log.** Dumped frames are written to `.cache/vmi-analysis/movie_frames/<run>/frame_0000.png`, `frame_0001.png`, and so on. A saved movie is `<output folder>/<dataset folder name>.mp4` (or `.gif`). The console reports the fixed display scale, the dump location, stream progress (when saving is off), and the saved movie path. In live mode the page loops the frames and shows *frame N / total*.

**Pitfalls.** The movie name comes from the dataset folder's basename, so two datasets with the same folder name overwrite each other. Unlike GMD Map and Dark Frame, this panel enumerates *all* `*.h5` files, so extra files change frame ordering. The colour maximum comes from the first `Scale frames` frames only; later brighter frames saturate. Frame dumps are always written, even when saving a movie is disabled.

## 5.3 Dark Frame

**Purpose.** Compute a mean or median dark frame from a folder of dark HDF5 files and save it as `dark_{method}.npy`, so downstream tools can reuse it instead of recomputing. The panel launches `vmi_analysis.centroiding.compute_dark`.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Dark HDF5 directory* | folder, empty | `dark.dark_dir` | *--dark-dir* | Folder of dark acquisitions. Required; an empty value makes the job fail immediately. |
| *HDF5 pattern* | text, `RAW-*.h5` | `dark.pattern` | *--pattern* | Glob used inside the dark folder; files are sorted by name. |
| *Frame dataset* | text, `raw/data` | `dark.dataset` | *--dataset* | Frame stack path inside each HDF5 file. |
| *Method* | select, `median` | `dark.method` | *--method* | `mean` uses a streaming running sum (low memory); `median` loads the selected frames and works in parallel (high memory). The method is part of the output file name. |
| *Dark frame count (0 = all)* | number, 2000 | `dark.frames` | *--frames* | Number of frames to use; 0 means every frame of every matched file. With a positive value the file list is trimmed using the first file's frame count. |
| *Output directory* | folder, workspace `centroiding/` | `dark.out_dir` | `--output-dir` | Where the `.npy` and its info file are written; created if missing. |

**Buttons.** *Run/Stop* submits job *Dark frame*. `Clear output` deletes `dark_<method>.npy` and

`dark_<method>_info.txt` for the *currently selected* method only.

**Results and log.** The results show `Dark frame: <npy> -- mean=..., std=..., min=..., max=...`, a grayscale preview, and the contents of the info file. Outputs are `<output dir>/dark_<method>.npy` (float64) and `dark_<method>_info.txt` with the method, frame count, statistics, and computation time. The console reports the number of files found, progress and rate, and per-file read warnings.

**Pitfalls.** Every run recomputes and overwrites. There is no shape check against the science data; the dark frame must match the camera frame shape used downstream. `median` can consume a lot of memory. Switching the method leaves the other method's files in place.

## 5.4 Accumulate

**Purpose.** Sum thresholded, morphologically denoised frames into one image, optionally weighting each frame by GMD pulse energy. This is a quick-look tool; the Picasso path in Centroiding is the precise one. The panel launches `vmi_analysis.imaging.h5_accumulate_fast_gmd`, and also hosts the frame-histogram preview for choosing the threshold (Section 5.4.1).

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *HDF5 directory* | folder, empty | `accumulate.h5_dir` | *--h5-dir* | Folder of `*.h5` files, sorted by name; every frame is used. Also read by *Histogram frame*. Required. |
| `Output image (.txt)` | save, empty | `accumulate.output_path` | *--output-path* | Accumulated matrix (text) and the base name for the script's PNG. Unlike the CLI, the GUI has no default path: a blank value fails only after the whole computation, so set it first. |
| *Threshold* | number, 5 | `accumulate.im_threshold` | *--im-threshold* | Pixels with value $\geq$ threshold are kept, after optional dark subtraction. Ignored in simple-sum mode. |
| *Workers* | number, 6 | `accumulate.max_workers` | *--max-workers* | Worker processes; the program caps them at the number of files. |
| *Denoise (2×2 opening)* | checkbox, on | `accumulate.do_denoise` | *--do-denoise* / `--no-do-denoise` | Applies a 2×2 morphological opening to each threshold mask before summing, removing isolated pixels. Ignored in simple-sum mode. |
| *Simple sum (no threshold)* | checkbox, off | `accumulate.simple_sum` | *--simple-sum* | Sums all frames unchanged: no threshold, no denoise. Dark subtraction and GMD weighting still apply. |
| *Frame index (0-based)* | number, blank | – | none (GUI preview) | Global 0-based frame for the histogram preview; blank chooses a random frame. Not saved and not sent to the accumulation job. |
| *GMD weighting* | checkbox, off | `accumulate.enable_gmd` | *--enable-gmd* | Enables per-frame GMD weights. The map and weight range are still sent when off, but the program ignores them. |
| *GMD map file* | file, workspace `gmd_map_all.txt` | `accumulate.gmd_map_path` | `--gmd-map-path` | Map produced by the GMD Map panel. Only the frame index and GMD value 1 columns are used. |
| *Weight min* | number, 0.0 | `accumulate.gmd_w_min` | *--gmd-w-min* | Lower weight bound; frames below it get weight 0 (discarded). |
| *Weight max* | number, 4.0 | `accumulate.gmd_w_max` | *--gmd-w-max* | Upper weight bound; frames above it get weight 0. The filter is active when *either* bound is above 0, so the default pair `does` filter. Set both to 0 to disable filtering. |
| `Dark frame .npy (optional)` | file, empty | `accumulate.dark_path` | *--dark-path* | Subtracted from every frame before thresholding and weighting; also used by the histogram preview. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Colormap* | select, Viridis | `accumulate.colormap` | none (display) | Colour scale for the interactive image and live preview. Two selects (above the live figure and above a saved result) share this one saved key. |

**Buttons.** *Run/Stop* submits job *Accumulate*. `Histogram frame` renders the single-frame histogram described in Section 5.4.1. *View saved result* re-renders the file at the current output path without rerunning. *Clear output* deletes the `.txt` and its `.png` so the next run recomputes. `open containing folder` and a *Saved PNG* expansion appear with results.

**Results and log.** While running, a live log-scale image is updated from a partial `.npy` written every 25 finished files (small datasets may show none). On success the results show the interactive accumulated image with the *Colormap* selector and hover counts. Outputs are the chosen `.txt`, its sibling `.png`, and the disposable partial file `.cache/vmi-analysis/accum_live_*.npy`. The console reports the dark-frame status, GMD weighting status, the weight filter range, the processing rate and the sum of counts.

**Pitfalls.** *Resume skip:* if the output `.txt` already exists, the program skips the accumulation and only replots (*[Resume] … skipping accumulation*); use *Clear output* or a new path to force a recomputation. GMD weighting corrects yield, never image shape. Frames missing from the map keep weight 1.0, so a partial map built with *First N files* silently leaves later frames unweighted. Threshold units are raw ADC counts unless a dark frame is set, in which case they apply to dark-subtracted values.

### 5.4.1 The frame histogram preview

The *Frame index (0-based)* field and *Histogram frame* button sit directly under the threshold row. Clicking the button reads one raw frame and plots its pixel-value histogram; it does not start a job, write a file, or take a concurrency slot.

- **Frame selection.** Leave the index blank for a uniformly random frame, or enter a global 0-based index. Frames are enumerated across all `*.h5` files, sorted by file name, in file order, exactly like the Movie tool. The caption shows *Frame k of N - file frame i (0-based index j)*. An out-of-range index is reported as an error.
- **What is plotted.** A 160-bin histogram of the frame's pixel values with a logarithmic count axis and a dashed line at the current *Threshold* value. The x axis is *Pixel value (counts)*.
- **Readouts.** The summary gives the number and percentage of pixels at or above the threshold, the frame median and maximum, and, when *Denoise (2×2 opening)* is checked, the number of pixels that survive the actual 2×2 opening. That last number is the most useful one: it is how much signal the combination of threshold and denoise keeps per frame.
- **Dark frames.** When a dark `.npy` is set on the panel, the preview subtracts it before computing the statistics, exactly like the accumulation run, and the caption says *dark-subtracted*.
- **Refresh.** Change the threshold or the frame and click again. *Hide* collapses the histogram.

The preview always applies the threshold statistic, even when `Simple sum` is checked; the checkbox only affects the accumulation job. The frame index is not part of the saved settings and resets when the page is reloaded.

## 5.5 Centroiding

**Purpose.** Localise electron spots in every frame with Picasso, build the 2D VMI histogram, optionally invert it, and produce the photoelectron spectrum. The panel launches `vmi_analysis.centroiding.process_andor_vmi_PICASSO_v5_h5_batch` with the working directory

`centroiding/`. Settings are saved automatically: each edit is stored immediately, and Run validates a snapshot before submitting, so an invalid value blocks the run but stays visible in the form.

The form has six groups: *Paths, detection and fitting* (21 fields), `Processing` (12), *Histogram* (2), *Energy calibration / Xe 4d fit* (4), *Summary spectrum plot axes* (4), and *GMD normalization* (3). Every field below is also a saved key prefixed with `centroid.`; the table shows the remainder.

### Paths, detection and fitting

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *HDF5 directory* | folder, empty | `vmi_picasso.h5_dir` | *--h5-dir* | Dataset folder; combined with the file pattern to find frames. Required: a blank value fails with a validation error. Relative paths resolve from `centroiding/`. |
| *HDF5 pattern* | text, `RAW-*.h5` | `vmi_picasso.h5_pattern` | *--h5-pattern* | Glob inside the directory; files are processed in sorted order and the global frame index runs across all of them. |
| *Frame dataset* | text, `raw/data` | `vmi_picasso.h5_dataset` | *--h5-dataset* | Frame stack inside each HDF5 file, expected `(N, H, W) uint16`. Also read from dark files. |
| *Metadata dataset* | text, `raw/meta` | `vmi_picasso.h5_meta` | *--h5-meta* | Kept for command-line compatibility; the current script does not read metadata. GMD values always come from the map file. |
| *GMD map file (empty = raw/meta)* | file, empty | `vmi_picasso.gmd_map_path` | `--gmd-map-path` | Frame-to-GMD table, used only when GMD weighting is on. The label is misleading: an empty or missing file produces a warning and all event weights become 1.0; there is no fallback to `raw/meta`. |
| *Dark HDF5 directory* | folder, empty | `vmi_picasso.dark_data_dir` | *--dark-data-dir* | Dark acquisitions, used only when dark subtraction is on and no usable pre-computed dark file exists. |
| *Dark pattern* | text, `RAW-*.h5` | `vmi_picasso.dark_h5_pattern` | *--dark-h5-pattern* | Glob inside the dark folder; ignored unless the dark folder is used. |
| `Pre-computed dark .npy` | file, empty | `vmi_picasso.dark_file` | *--dark-file* | Highest-priority dark frame, for example one produced by the Dark Frame panel. Relative paths are joined to `centroiding/`. |
| *Output directory* | folder, `vmi_output` | `vmi_picasso.output_dir` | *--output-dir* | Directory for every output (created if missing). A relative value is resolved under `centroiding/`, so the default is `centroiding/vmi_output`. |
| *Checkpoint filename* | text, `vmi_results_raw.h5` | `vmi_picasso.checkpoint_file` | `--checkpoint-file` | Name inside the output directory. If the file exists, the batch *skips localisation* and rebuilds the histogram and summary from it (see the pitfalls). |
| *Box size (odd)* | number, 5 | `vmi_picasso.box_size` | *--box-size* | Picasso box side in camera pixels: local-maximum search and fit cut-out. Must be an odd integer of at least 3. Larger tolerates larger spots but mixes neighbours and slows fitting. |
| *Min net gradient* | number, 400.0 | `vmi_picasso.min_net_gradient` | *--min-net-gradient* | Detection threshold passed to Picasso's `identify`, in camera counts. Too high loses dim events; too low floods the fit with noise. The tuning check plots its histogram with this value marked. |
| *Camera baseline* | number, 0.0 | `vmi_picasso.camera_baseline` | *--camera-baseline* | ADU offset removed in the counts-to-photons conversion. Independent of dark subtraction. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Camera sensitivity* | number, 1.0 | `vmi_picasso.camera_sensitivity` | *--camera-sensitivity* | Electrons per ADU used in the photon conversion. |
| *Camera gain* | number, 1.0 | `vmi_picasso.camera_gain` | *--camera-gain* | Gain divisor in the same conversion; use 1.0 for a non-multiplying sensor. |
| *Camera QE* | number, 0.9 | `vmi_picasso.camera_qe` | *--camera-qe* | Passed to Picasso's camera information. With the Picasso version currently pinned by this project the quantum-efficiency term is not applied, so the value has no effect on results. |
| *MLE convergence* | number, 0.1 | `vmi_picasso.mle_convergence` | *--mle-convergence* | MLE fitting only: iteration stops when the fitted position change falls below this many pixels. Lower is slower and stricter. |
| *MLE max iterations* | number, 50 | `vmi_picasso.mle_max_iterations` | *--mle-max-iterations* | MLE only: hard iteration cap. |
| *MLE method* | select, `sigma` | `vmi_picasso.mle_method` | *--mle-method* | MLE only: `sigma` fits one shared width; `sigmaxy` fits independent horizontal and vertical widths. |
| *Fitting method* | select, `mle` | `vmi_picasso.fitting_method` | *--fitting-method* | `mle` is Poisson-optimal and uses the three MLE fields; `lq` is a faster least-squares fit that ignores them. Both discard fits with a width of 0.3 pixels or less. |
| *Tuning frame index* | number, 0 | `vmi_picasso.tuning_frame_index` | *--tuning-frame-index* | Global 0-based frame used by the tuning check; clamped to the last available frame. |

## Processing

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Parallel processing* | checkbox, on | `processing.enable_parallel` | *--parallel* / `--no-parallel` | One worker per HDF5 file. Each file must still fit in memory. Off runs sequentially. |
| *CPU cores* | number, 6 | `processing.num_cores` | `--num-cores` | Worker-pool size; must be at least 1. It is sent even when parallel processing is off. |
| *Dark subtraction* | checkbox, off | `processing.enable_dark_subtraction` | *--dark-subtraction* / `--no-dark-subtraction` | Master switch for dark correction. When no dark file or folder is usable the script falls back to a per-frame clipped median background. |
| *Dark method* | select, `mean` | `processing.dark_method` | *--dark-method* | How a dark frame built from the dark folder is combined: streaming mean (low memory) or median (all selected frames in memory). Ignored when a pre-computed `.npy` is used. |
| *Dark frame count* | number, 2000 | `processing.dark_frame_count` | *--dark-frame-count* | Frames used for the computed dark; must be at least 1. Unlike the Dark Frame panel, 0 is *not* "all frames" here. |
| *Flat field* | checkbox, off | `processing.enable_flat_field` | *--flat-field* / `--no-flat-field` | Enables per-pixel flat correction (divide by the flat normalised to its mean). |
| *Flat field file* | file (.txt/.npy), empty | `processing.flat_field_file` | *--flat-field-file* | Flat image. The loader expects a TIFF despite the picker filter, so convert the flat to TIFF before use. |
| *Circular mask* | checkbox, off | `processing.enable_circular_mask` | *--circular-mask* / `--no-circular-mask` | Present in the form and recorded in the summary, but the current script does not apply any mask to events or images. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Mask center X (empty = image center)* | number, empty | `processing.mask_center_x` | `--mask-center-x` | Mask geometry, currently unused. |
| *Mask center Y (empty = image center)* | number, empty | `processing.mask_center_y` | `--mask-center-y` | Mask geometry, currently unused. |
| *Mask radius (empty = automatic)* | number, empty | `processing.mask_radius` | `--mask-radius` | Mask geometry, currently unused. |
| *Tuning mode only* | checkbox, off | `processing.tuning_only` | *--tuning-only* / `--no-tuning-only` | Runs the single-frame tuning check and stops before the batch (see below). The Pipeline panel forces this off in its own snapshot without changing the saved form. |

## Histogram, energy calibration, spectrum axes and GMD weighting

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Histogram resolution factor* | number, 1 | `histogram.resolution_factor` | *--resolution-factor* | Upscales the 2D histogram and the event coordinates by this factor. Must be at least 1. Alpha is defined in `histogram` pixels, so changing this factor invalidates an old alpha. |
| *Abel inversion* | checkbox, on | `histogram.enable_abel` | *--abel* / `--no-abel` | Enables the PyAbel inversion, the photoelectron spectrum, the CSV, and the Xe 4d fit. When off there is no `vmi_3d_abel.npy` and no spectrum. |
| *$\alpha$ (eV/pixel$^2$; empty = uncalibrated)* | number, empty | `energy_calibration.alpha` | `--alpha` | Converts the spectrum x axis with $E = \alpha r^2$. Empty keeps the squared-radius axis in pixel$^2$. Must be positive when set. Intensity is never divided by alpha. |
| *Fit Xe 4d doublet -> $h\nu$ and $\alpha$ in console* | checkbox, off | `energy_calibration.fit_xe_4d` | `--fit-xe-4d` / `--no-fit-xe-4d` | Fits two Gaussians plus a linear background to the two strongest peaks in $r^2$ and prints suggested alpha and photon energy. Advisory only: it changes neither alpha nor the plotted coordinates. Requires Abel inversion. |
| *Xe fit min (pixel$^2$; empty = auto)* | number, empty | `energy_calibration.fit_min_px2` | `--fit-min-px2` | Lower bound of the fit window in pixel$^2$. Use it to isolate the doublet from other lines. |
| *Xe fit max (pixel$^2$; empty = auto)* | number, empty | `energy_calibration.fit_max_px2` | `--fit-max-px2` | Upper bound of the fit window; same rules. |
| *Spectrum x min (-1 = auto)* | number, $-1$ | `spectrum_plot.x_min` | *--spectrum-x-min* | Left limit of the summary spectrum panel only (not the CSV). Negative means automatic. Units follow alpha. |
| *Spectrum x max (-1 = auto)* | number, $-1$ | `spectrum_plot.x_max` | *--spectrum-x-max* | Right limit; zero or negative means the last energy. |
| *Spectrum y min (-1 = auto)* | number, $-1$ | `spectrum_plot.y_min` | *--spectrum-y-min* | Bottom limit; negative means zero. |
| *Spectrum y max (-1 = auto)* | number, $-1$ | `spectrum_plot.y_max` | *--spectrum-y-max* | Top limit; zero or negative means automatic. Reset the limits when switching between pixel$^2$ and eV. |
| *GMD weighting* | checkbox, off | `gmd.enable_gmd_weighting` | *--gmd-weighting* / `--no-gmd-weighting` | Weights each event by mean GMD divided by its frame's GMD. GMD is a per-frame scalar, so this corrects yield, never spectral shape. |
| *GMD weight min* | number, 0.0 | `gmd.gmd_weight_min` | *--gmd-weight-min* | Weights below this are zeroed when the value is above 0. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *GMD weight max* | number, 4.0 | `gmd.gmd_weight_max` | *--gmd-weight-max* | Weights above this are zeroed when the value is above 0. With the default, frames whose GMD is below a quarter of the mean contribute nothing. |

Four schema keys have no widget but are migrated and passed to the script: `meta_column_gmd`, `tuning_bins_photons`, `tuning_bins_net_gradient`, and `tuning_bins_precision`. They are unused or fixed by the schema, so they are not documented as user controls.

### Tuning mode versus full batch

`Tuning mode only` runs Picasso on one selected frame, writes the tuning figure and its data sidecar, and stops before localisation of the whole dataset, before the histogram, Abel inversion and summary, and without writing a checkpoint. The console ends with `TUNING MODE - Batch processing skipped`. Tuning is not a parameter scan: edit the detection fields and run again. The usual sequence is to tune one frame, then untick the checkbox and run the full batch.

A full batch first prepares dark and flat corrections, then localises (or resumes from an existing checkpoint), builds the 2D histogram, optionally inverts it and builds the spectrum, and finally writes the summary. Progress appears in the console (*Processing HDF5 files*, *Building histogram*).

### Results

After a successful run the results area shows two interactive Plotly figures when their data sidecars exist: *Tuning check* (corrected frame with localisation circles, photon distribution, net-gradient distribution with the threshold line, and localisation precision) and *Final summary* (raw VMI, Abel-inverted VMI, photoelectron spectrum, and a text block of run settings). Each has a colormap selector (*Tuning colormap*, *Raw VMI colormap*, *Abel VMI colormap*) and a `Saved PNG` expansion for the original figure. Hover for values, drag to zoom, double-click to reset. If only the PNG exists, the panel says so and asks you to rerun once to create the interactive data. *View saved results* rebuilds these views from the current output directory without rerunning; *Clear outputs* deletes the listed files after confirmation, including the checkpoint if its checkbox is ticked.

The final summary axes are *Squared radius (pixel$^2$)* or `Kinetic energy (eV)` depending on whether alpha was set at run time; the y axis is always *Intensity (arb. units)*.

### Files written

| File | Written when | Contents |
|---|---|---|
| `vmi_results_raw.h5` | full batch, kept on resume | Localisation checkpoint and archive: one row per event with position, photons, size, eccentricity, signal, net gradient and global frame index. Downstream binning reads this file. |
| `vmi_2d_raw.npy` | full batch | 2D VMI histogram (weighted if GMD weighting is on). |
| `vmi_3d_abel.npy` | full batch with Abel | Centred and rotated Abel-inverted distribution. |
| `energy_spectrum.csv` | full batch with Abel | Columns `r_px,r2_px2,energy,dN_per_dr,intensity`; energy is $\alpha r^2$ or $r^2$ when alpha is empty. |
| `04_final_summary.png` / `.npz` | full batch | The 2×2 summary figure and its interactive data (metadata plus arrays). |
| `01_tuning_check_picasso.png` / `.npz` | tuning mode only | The four-panel tuning figure and its data (frame, positions, photons, net gradient, precision). |

The constants `vmi_results.h5` and `vmi_results.csv` exist in the script but are never written; do not expect them.

### Pitfalls

- **Resume skips localisation silently.** If the checkpoint file already exists, the batch skips Picasso and rebuilds the histogram and summary from it. Changes to box size, threshold or fitting method are ignored until the checkpoint is deleted (tick the checkpoint checkbox in *Clear outputs*).
- **Tuning artifacts are directory-scoped.** A full batch never overwrites the tuning figure, so *View saved results* can show a tuning check from an earlier parameter set.
- $\alpha$ **is per histogram pixel.** Recalibrate it whenever the histogram resolution factor changes.
- **The Xe fit is advisory.** It prints suggested values but never changes alpha or the coordinates; with Abel off it does not run at all.
- **Unused controls.** The metadata dataset, circular mask and its centre/radius fields, and camera QE currently have no effect on the numerical result.
- **Flat-field format mismatch.** The picker offers `.txt`/`.npy`, but the loader reads TIFF images.
- **Empty numeric values.** Empty spectrum limits become −1 (automatic), empty alpha and mask values stay empty, and other empty numbers become 0, which often fails validation (for example CPU cores).

## 5.6 Binning

**Purpose.** Partition the localised events into bins of GMD pulse energy and build one 2D histogram per bin. The panel launches `vmi_analysis.centroiding.vmi_histogram_gmd_bins` with working directory `centroiding/`; run it after Centroiding.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| `Localization checkpoint (.h5)` | file, the Centroiding output's `vmi_results_raw.h5` | `binning.checkpoint` | *--checkpoint-path* | Checkpoint written by the Centroiding batch. A missing file fails with *Checkpoint not found*. |
| *Output directory* | folder, the Centroiding output directory | `binning.output_dir` | `--output-dir` | Where the per-bin `.npy` files and the overview PNG are written. |
| *Frame-to-GMD map* | file (.txt), migrated from Centroiding | `binning.gmd_map` | `--gmd-map-path` | The map produced by GMD Map. A missing file fails with *GMD map not found*. |
| *Histogram resolution factor* | number, 1 | `binning.resolution_factor` | `--resolution-factor` | Multiplies event coordinates and histogram size. It must match the Centroiding value, otherwise alpha means a different energy per pixel. |
| *GMD thresholds (comma-separated)* | text, `3.75e-8,5.5e-8,7.25e-8` | *binning.thresholds* | `--thresholds` | Ascending GMD cut values. $N$ thresholds define $N+1$ bins: below the first, between successive values, and above the last. The values are not sorted or validated. |
| *GMD column (4=m1, 5=m2)* | select, 4 | `binning.gmd_col` | *--gmd-col* | Which GMD value column of the map to use: 4 for method 1, 5 for method 2. |
| *Weight mode* | select, `gmd_norm` | `binning.weight_mode` | *--weight-mode* | `none`: every event counts 1. `gmd`: weight equals the frame's GMD value. `gmd_norm`: weight equals mean GMD divided by the frame's GMD, raised to *Weight power*. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Weight power* | number, 1.0 | `binning.weight_power` | *--weight-power* | Exponent for `gmd_norm`. Linear (1.0) over-corrects for the reference dataset; an effective exponent near 0.42 flattened the yield. |
| *GMD floor* | number, 1e-12 | `binning.gmd_floor` | *--gmd-floor* | Non-positive GMD values are raised to this before binning and weighting, avoiding division by zero. |
| *GMD weight min* | number, 0.0 | `binning.gmd_weight_min` | *--gmd-weight-min* | In `gmd_norm` mode, computed weights below this are zeroed when the value is above 0. |
| *GMD weight max* | number, 4.0 | `binning.gmd_weight_max` | *--gmd-weight-max* | Weights above this are zeroed when the value is above 0. The default discards frames whose GMD is below a quarter of the mean. |

**Bin and weight semantics.** A frame's bin index is the number of thresholds below its GMD value, so the first bin has no lower bound and the last has no upper bound. Events from frames beyond the end of the map are dropped and counted as *Events w/o GMD*. Events whose frame GMD is NaN are placed in the *highest* bin by the current implementation (despite an in-code comment claiming they are discarded), so inspect a map with gaps before relying on the last bin. The overall histogram is the sum of the per-bin histograms, so it is weighted when a weighting mode is active.

**Outputs.** One file per bin, `vmi_2d_gmd_bin<b>_<lo>_<hi><suffix>.npy`, where `<lo>`/`<hi>` are `minusinf`, `plusinf` or a threshold; the summed file `vmi_2d_gmd_all<suffix>.npy`; and the overview figure `vmi_gmd_bins<suffix>.png`. The suffix encodes the weighting: empty for `none`, `_w-gmd` for `gmd`, `_w-gmd_norm` for `gmd_norm` with power exactly 1, and `_w-gmd_norm_p<value>` otherwise. The results area displays the overview PNG.

**Pitfalls.** The histogram size is hardcoded to 1024×1024 camera pixels and the checkpoint's real image shape is not read. The resolution factor is independent of the Centroiding one. Invalid thresholds (empty, unparseable, unsorted) are not caught by the panel. The weight minimum and maximum in this panel are a separate copy from the Centroiding tab's values; the Pipeline uses the Centroiding values instead. Binning writes its whole output directory, so it is queued against a concurrent Centroiding or Pipeline job writing the same directory.

## 5.7 Pipeline

**Purpose.** Run the whole chain as one ordered job: GMD map, Centroiding, Binning, and PES/Abel inversion. All stages run sequentially in one job; a failure stops the chain. Configure the Centroiding parameters on their own tab first, because the pipeline borrows those saved settings and combines them with its own dataset and GMD map.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *HDF5 directory (dataset)* | folder, empty | `pipeline.h5_dir` | stage 1 `--h5-dir` | Dataset for the whole pipeline; also forced into the Centroiding snapshot as an absolute path. Required. |
| *GMD timestamp file* | file, empty | `pipeline.gmd_path` | stage 1 `--gmd-path` | GMD table for stage 1. Required. |
| *GMD map output* | save, workspace `gmd_map_all.txt` | `pipeline.map_out` | stage 1 `--output-path` | Where the map is written; also passed to Centroiding and Binning. |
| *Weight mode* | select, `gmd_norm` | `pipeline.weight_mode` | Binning `--weight-mode` | Weighting for the Binning stage and the suffix used by later stages. |
| *Weight power* | number, 1.0 | `pipeline.weight_power` | Binning `--weight-power` | Exponent for `gmd_norm`. |
| *Skip stages whose outputs exist* | checkbox, on | `pipeline.skip_existing` | job-level | A stage is skipped when all of its declared output files already exist. |

**Stages.** The panel shows four status badges, *GMD map*, `Centroiding`, *Binning* and *PES*, updated while the job runs. Each stage has a fixed working directory and output fingerprint:

| Stage | Command | Working directory | Output fingerprint |
|---|---|---|---|
| GMD map | `build_gmd_map` with the three path fields | repository root | the map file |
| Centroiding | `process_andor_vmi_PICASSO_v5_h5_batch` with the full Centroiding snapshot | `centroiding/` | the checkpoint file |
| Binning | `vmi_histogram_gmd_bins` with the checkpoint, output directory, map and thresholds | `centroiding/` | `vmi_2d_gmd_all<suffix>.npy` |
| PES | `process_gmd_bins` with the weight suffix, bin directory and PyAbel backend | `circularization/` | `gmd_bins_results.npz` in the automatic output directory |

**Borrowed and forced settings.** The pipeline takes every Centroiding setting from the saved form, but overrides exactly three values in its own snapshot: the HDF5 directory, the GMD map path, and *Tuning mode only* (forced off, so a pipeline run is always a full batch and never produces a tuning check). Thresholds, GMD column and GMD floor are taken from the Binning tab's saved settings; the Pipeline fields supply the weight mode and power. The pipeline has no output-directory field: the Centroiding output directory is used. The PES stage always uses the PyAbel backend with the `daun` method; MATLAB is never selected from the pipeline.

**Status, resume and cancellation.** With *Skip stages whose outputs exist* checked, a stage whose outputs are all present is marked `skipped` and the log records it; this is the only resume mechanism. The first failing stage stops the pipeline, marks the remaining stages `cancelled`, and fails the job. Stopping cancels queued work immediately and terminates the running process tree. The pipeline reserves the inputs and outputs of all four stages for the whole run, so overlapping jobs wait.

**Pitfalls.** There is no results view: results are the stage logs and the files each stage wrote. The stage badges are not restored after a page reload (the job badge and console are). Deleting a stage's output is the way to force that stage to run again.

## 5.8 Circularize

**Purpose.** Correct the radial and azimuthal distortion of one VMI image using a Gascooke trig-fit calibration. The panel launches `vmi_analysis.circularization.circularization` with working directory `circularization/`. It is a standalone tool for inspecting the calibration. The PES panel applies its correction to each GMD bin.

For a raw detector-grid calibration, open the same `vmi_2d_raw.npy` in Trig Calibration and Circularize. Use the same origin in both panels. The default is the brightest pixel. The NumPy input keeps its native shape and orientation. Circularize applies the radial correction directly and writes the corrected image.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| `Input image (.npy or .txt)` | file, Centroiding raw image | *circularize.inp* | `--input` | NumPy detector array or legacy text image. |
| `Output image (.txt)` | save, `circularization/image_circularized.txt` | *circularize.out* | `--output` | Corrected image, written as text in scientific notation. |
| *Shape NY* | number, 1023 | `circularize.shape_y` | `--shape` (first) | Number of rows for legacy text input. NumPy input uses its native shape. |
| *Shape NX* | number, 1023 | `circularize.shape_x` | `--shape` (second) | Number of columns for legacy text input. |

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Use brightest pixel as center* | checkbox, on | `circularize.brightest_center` | — | For NumPy input, use the brightest pixel or clear the checkbox to enter a center. |
| *Center X* | number, 511.0 | `circularize.center_x` | `--center` (first) | Zero-based column of the manually specified origin. |
| *Center Y* | number, 511.0 | `circularize.center_y` | `--center` (second) | Zero-based row of the manually specified origin. |
| *Extracted trig-fit file* | file, `circularization/extracted-FOC100eV-20260710.txt` | `circularize.extracted` | *--extracted* | Gascooke export with one block per fitted ring: radius, trigonometric coefficients and uncertainties. Required. |
| *Reassignment file (optional)* | file, `circularization/reassignment-FOC100eV-20260710.txt` | `circularize.reassign` | *--reassign-file* | Two-column map from fitted to assigned radius. Both columns must increase together. Clear the field to use the identity map. |
| *Radial model* | select, `interp` | `circularize.radial_model` | *--radial-model* | `interp` uses shape-preserving interpolation of the coefficients; `poly` fits a polynomial that vanishes at the centre. |
| *Poly order* | number, 2 | `circularize.poly_order` | *--poly-order* | Degree of the polynomial. This field is enabled when the radial model is `poly`. |
| *Interp order* | number, 3 | `circularize.interp_order` | *--interp-order* | Spline order used for the inverse warp (3 = cubic). |
| *Max trig n (0 = auto)* | number, 0 | `circularize.max_trig_n` | *--max-trig-n* | Highest trigonometric order to include; 0 omits the flag and lets the script infer the order from the file. |
| *Polar dr* | number, 1.0 | `circularize.polar_dr` | *--polar-dr* | Cartesian pixels per polar radial unit for legacy text input. NumPy calibration uses detector pixels directly. |
| *No Jacobian* | checkbox, on | `circularize.no_jacobian` | *--no-jacobian* | Checked skips the Jacobian intensity correction. Note that the command-line default is the opposite (correction on). |
| *No diagnostic plots* | checkbox, off | `circularize.no_plots` | *--no-plots* | Skips the per-order trigonometric fit figures. A reassignment map figure is still written when a reassignment file is used. |
| *Summary only* | checkbox, off | `circularize.summary_only` | *--summary-only* | Parse the extracted file and print its summary, then stop before loading the image or writing outputs. |
| *Plot output dir* | folder, `circularization/trig_fit_plots` | `circularize.plot_dir` | `--plot-dir` | Where the diagnostic PNGs are written. |

**Results.** On success the panel shows the corrected image with a logarithmic colour scale and every diagnostic PNG from the plot directory, each openable in the zoom/crop overlay. The console prints the parsed rings, the available trigonometric series, the maximum radial contribution of each term, and the applied reassignment. Outputs are the corrected text image and the diagnostic PNGs; *reassignment_map.png* is written whenever a reassignment file is supplied.

**Pitfalls.** The calibration files are per-experiment inputs. The extracted trig-fit file can be built clean-room with the Trig Calibration tab; the reassignment file is supplied separately (identity is valid). The panel does not pre-check them, so a missing file fails the job; because the reassignment path is pre-filled, a missing reassignment file also fails. Clear that field to use the identity map. The GUI's *No Jacobian* default differs from the command-line default, which is the most consequential default difference in this panel. Beyond the last assigned radius, reassignment keeps the final radial offset

constant. Circularize checks the combined warp for invalid radii and folds on the output grid before resampling. There are no `View saved result` or *Clear output* buttons, and the progress bar stays empty because the script does not print percentages.

## 5.9 Trig Calibration

**Purpose.** Build the extracted trig-fit calibration from a VMI image without the Gascooke executable. The panel remaps the image to polar coordinates ($x$ = radius, $y$ = angle, the orientation of the Gascooke *Theta vs r* image) and optionally applies inverse Abel to each constant-angle line. The remap samples detector intensity at each polar coordinate and preserves that intensity without a $1/r$ correction. The fitter uses the processed polar data to estimate a Gaussian ridge whose centre varies as $r_g(\theta) = r_0 + \sum_n [A_n \sin(n\theta) + B_n \cos(n\theta)]$. Accepted rings are appended to a trig-fit file in the format consumed by the Circularize and PES panels. This tab is an interactive tool: it submits no job and launches no external program. Selection is manual by design; there is no automatic ridge finder.

| Field | Type and default | Saved key | Meaning |
|---|---|---|---|
| `VMI image` | file, Centroiding raw image | `calibration.image` | 2D NumPy detector image (`.npy`) in its original orientation. |
| *Use brightest pixel as center* | checkbox, on | `calibration.brightest_center` | Use the brightest pixel as the polar origin. Clear the checkbox to enter the origin manually. |
| *Centre X*, *Centre Y* | numbers, 511.0 | `calibration.center_x`, `.center_y` | Zero-based column and row of the fit origin in the detector image. |
| *Radial bin size (px)* | number, 1.0 | `calibration.radial_bin_size` | Spacing between radial samples in detector pixels. Sampling starts at $r = 0$ and stops at the last grid point at or below the maximum radius. A spacing of 0.5 samples the image by interpolation every half pixel in radius. |
| *Angular bins* | number, 720 | `calibration.n_theta` | Number of angular samples over one full turn. |
| *Max radius (0 = auto)* | number, 0.0 | `calibration.r_max` | Largest radius to sample; 0 uses the largest circle inside the image. |
| *Inverse Abel per θ* | checkbox, on | `calibration.abel` | Invert each constant-angle radial line to locate ridges. For anisotropic images this is a ridge-finding approximation. |
| *Abel method* | select, `hansenlaw` | `calibration.abel_method` | Hansen–Law (the method used by the reference software); Dasch three-point, two-point and onion-peeling are available for comparison. |
| *Filter placement* | select, `post_abel` | `calibration.placement` | With inverse Abel enabled, choose *After Abel inversion* or *Off*. With inverse Abel disabled, choose *Raw polar* or *Off*. *Off* skips filtering. |
| *Enhancement* | select, `bandpass` | `calibration.enhance_mode` | Choose `none`, `bandpass`, `ridge`, `unsharp` or `clahe`. |
| *Filter parameters* | expansion | `calibration.param.*` | Editable defaults: smooth $\theta$ 1.0, smooth $r$ 0.8; bandpass low $r$ 2.0 and high $r$ 20.0; ridge background 8.0/25.0 and ridge $\sigma$ 1.2; unsharp blur 6.0/18.0; CLAHE clip 0.02 and kernel 24/96. |
| *Select band / Mask* | toggle | — | Drag with the left mouse button: a band (blue) selects the fit region, a mask (red) excludes interference. |
| *Full angular range* | checkbox, on | `calibration.full_angle` | The band spans 0–360°; untick to use the drawn angular extent. |
| *Trig terms N* | number, 3 | `calibration.n_terms` | Highest harmonic order fitted in $r_g(\theta)$. |
| *Fit beta* | checkbox, on | `calibration.beta` | Fit the $P_2(\cos\theta)$ anisotropy amplitude. |
| *Iterative terms* | checkbox, on | `calibration.iterative` | Fit $1, 2, \ldots, N$ terms cumulatively instead of all at once. |
| *Extracted trig-fit file* | save, `circularization/extracted-YYYYMMDD.txt` | `calibration.output` | Accepted rings are appended here; create a fresh file per calibration. |
| *Provenance comments* | checkbox, on | `calibration.provenance` | Write `#` comment lines before each block (band, filter, Abel method, RMS). Parsers ignore them. |

| Field | Type and default | Saved key | Meaning |
|---|---|---|---|
| *Radial model* | select, Interp | `calibration.radial_model` | Use shape-preserving cubic interpolation (Interp) or a through-origin polynomial (Poly) to model $A_n(r)$ and $B_n(r)$. |
| *Poly order* | number, 1 | `calibration.slope_order` | Degree of the through-origin polynomial. Order 1 gives a straight line. |
| *Session file* | save, `circularization/trig-calibration.session.json` | `calibration.session` | Save/load image geometry, processing, masks and accepted ring fits. |

**Workflow.** (1) Choose the image and click *Load image & remap*. For Centroiding data, use its raw output `vmi_2d_raw.npy`. The console reports the origin and radial spacing. The polar remap uses this origin on the original detector grid. (2) Set inverse Abel and the display filter on their separate rows, then press *Apply processing*. The processed preview appears directly below the raw polar remap. Both show the same band and masks. The fit uses the selected region of the processed image, including any applied filter. (3) With *Select band* active, drag a rectangle around one ring. *Zoom to band* zooms along the radius while keeping the angular view fixed. *Inspect band* opens the clipped processed band at the same display height. Use horizontal zoom from 0.05 to 1 to narrow the band and draw masks that appear on both previews. *Color scale*, *Colormap*, and the percentile sliders update both previews as you adjust them. The sliders span 0 to 100 in steps of 0.1 and default to the 1st and 99.5th percentiles. Each preview computes its limits from the visible pixels. Log scaling uses only positive pixels. (4) Switch to *Mask* and drag rectangles over overlapping rings, hot spots or bad sectors. *Undo selection* reverses the last band or mask edit on both previews. Use *Clear band* to remove the blue band or *Clear masks* to remove all red masks. Undo also reverses either clear action. (5) Press *Fit selection*. The comparison shows the processed band used for the fit and the model with shared axes and intensity scale. Grey regions mark excluded pixels, and the yellow line shows $r_g(\theta)$. Review the images before accepting the ring. (6) Press *Accept ring* to append one block to the output file, then repeat from step 3 for the next ring. (7) Choose a radial model and use *Update radial model plots* to compare its curves with the accepted ring measurements. Each harmonic uses the rings that measured it. In Poly mode, each term needs at least as many distinct measured radii as the polynomial degree. The panel reports terms fitted without weights when some uncertainties are missing or invalid. Click either *Send to Circularize* button to pass the NumPy image, center, calibration file, and radial model to Circularize. The export includes the polynomial degree when Poly is selected. Use *Refresh rings from file* to replace the accepted-ring list and plots with the contents of the selected calibration file. Use *Clear calibration file* to delete that file and reset the ring list and plots after confirmation.

**Results.** The panel shows both polar previews with band, masks and fitted ridge overlays, a fit comparison, the parameter table, the accepted-ring table, and separate $A_n(r)$ and $B_n(r)$ plots with ring measurements and their uncertainties. The parameter table reports local least-squares errors. These estimates do not account for correlations between interpolated pixels. *Rebuild file from rings* rewrites the file from the accepted rings and is the recovery path after a wrong or duplicate append.

The optimizer can report a warning while still returning a model for review. If the data cannot determine some parameters, widen the selected angular range or reduce *Trig terms N* and refit. The panel blocks export when the model is underdetermined or has nonfinite parameters.

**Pitfalls.** Enhancement can move the fitted ridge. Use consistent processing settings for the rings in a calibration. Refit rings saved by the earlier fitting pipeline and export them to a new calibration file. Calibrate in the same orientation and centre frame as the images you will correct: PES rotates and centres each bin before it applies the calibration. Hansen–Law assumes the first radial column is $r = 0$; keep the default radial grid. Masks, processing settings and provenance live in the session file; the extracted `.txt` contains only the ring blocks. A ring whose radius is already present within 0.5 px is refused to avoid duplicates. The file is append-only; use *Rebuild* to rewrite it. Circularize sorts ring blocks by radius before building the interpolant and computes its radial model from the saved blocks.

## 5.10 PES / Abel inversion

**Purpose.** Process every GMD-binned image through hole patching, rotation, centring, circularisation, apodisation and Abel inversion, and produce the photoelectron energy spectrum and angular-anisotropy parameters for each bin. The panel launches `vmi_analysis.circularization.process_gmd_bins` with working directory `circularization/`.

| Field | Type and default | Saved key | CLI flag | Meaning |
|---|---|---|---|---|
| *Weight suffix* | text, `_w-gmd_norm` | `pes.weight_suffix` | *--weight-suffix* | Selects which bin files to process: files whose name ends with this suffix. Empty means unweighted bins. |
| *Bin image directory* | folder, saved Centroiding output directory | `pes.bin_dir` | `--bin-dir` | Directory holding the `vmi_2d_gmd_bin*.npy` files from Binning. |
| *Output directory (empty = automatic from suffix/ backend)* | folder, empty | `pes.output_dir` | `--output-dir` | Where results are written. Empty means `circularization/gmd_bins_results<suffix>`, plus `_pyabel_<method>` for the PyAbel backend. |
| *Inversion backend* | select, `pyabel` | `pes.backend` | *--inversion-backend* | `pyabel` is pure Python; `matlab` uses the bundled CPBASEX code and requires MATLAB plus a generated basis file. |
| *PyAbel method* | select, `daun` | `pes.method` | *--pyabel-method* | Inversion algorithm. Options: `daun`, `hansenlaw`, `three_point`, `two_point`, `basex`, `rbasex`, `linbasex`, `onion_bordas`, `direct`. |
| *Use cached inversions* | checkbox, off | `pes.use_cache` | *--use-cache* | Reuse a bin when its four cached files exist: energy, intensity, beta and cleaned-image text files. Partial caches mix reused and recomputed bins. |

**Per-bin chain.** For each selected bin the program repairs the detector hole with a feathered patch, rotates by 90 degrees, centres the image (cropping 1024×1024 to 1023×1023), runs the circularisation subprocess, centres again and applies a cosine-squared edge apodisation, then inverts the image. The PyAbel path uses the selected method and extracts the energy axis, the intensity and $\beta_2$ to $\beta_{10}$. The MATLAB path writes the cleaned image to the CPBASEX shared input, runs the MATLAB script, and immediately copies its outputs out before the next bin; this is why bins always run sequentially. Several values are fixed in the script and not exposed by the panel: the old and new centres, the hole radius and donor offsets, the apodisation cutoff and taper, the trigonometric order, the interpolation order, and the alpha used for the energy axis.

**Outputs.** Per bin, text files for the energy axis, intensity spectrum, beta parameters and the cleaned image, all named `<name>_GMD_<label>.txt`. After all bins, three overview figures (`gmd_bins-energy-spectrum.png`, `gmd_bins-energy-spectrum-normalized.png`, `gmd_bins-betas.png`) and `gmd_bins_results.npz` containing the bin labels and the arrays. The results area shows the three overview figures in that order, with a button to open the results folder.

**Calibration and backend requirements.** Both calibration files (Section 4.4) must exist in `circularization/`; otherwise the job fails before processing. The MATLAB backend additionally needs the basis file `G_r512_k512_l10.h5`, generated once with `save_gData.m` in MATLAB (time-consuming). The panel warns when the calibration files are missing and when the MATLAB basis is absent. The default PyAbel backend needs neither MATLAB nor the basis file.

**Pitfalls.** The panel has no *View saved result* or `Clear output` buttons, so after a reload past results cannot be reopened from the interface. The progress bar stays empty because the script does not print percentages. Individual bin failures are reported and processing continues; if no bin succeeds the job fails. Weighted and unweighted runs write to different automatic output directories so their caches cannot collide.

# 6 Output files

Every analysis stage writes into the output directory you select. The table summarises the files by stage; the panel sections give the details.

| Stage | Files | Notes |
|---|---|---|
| GMD Map | `gmd_map_all.txt` | Two header lines, then one row per frame: global frame index, file name, frame in file, GMD index, two GMD values, and signed time difference in milliseconds. |
| Centroiding | `vmi_results_raw.h5`, `vmi_2d_raw.npy`, `vmi_3d_abel.npy`, `energy_spectrum.csv`, `04_final_summary.png/.npz`, `01_tuning_check_picasso.png/.npz` | The checkpoint is both the resume file and the archive of localised events. The tuning files are written only by a tuning run. The CSV exists only when Abel inversion succeeds. |
| Binning | `vmi_2d_gmd_bin<b>_<lo>_<hi><suffix>.npy`, `vmi_2d_gmd_all<suffix>.npy`, `vmi_gmd_bins<suffix>.png` | One 2D histogram per GMD bin, the sum of the bins, and an overview figure. The suffix encodes the weighting. |
| PES | `<name>_GMD_<label>.txt`, three overview PNGs, `gmd_bins_results.npz` | Per-bin energy, intensity, beta and cleaned-image text files; overview spectra and beta figures; one archive with all arrays. |
| Dark Frame | `dark_mean.npy` or `dark_median.npy`, `dark_{method}_info.txt` | Reusable dark frame and its statistics. |
| Accumulate | `<output>.txt`, `<output>.png` | Accumulated matrix and its log-scale figure. |
| Circularize | `image_circularized.txt`, diagnostic PNGs | Corrected text image; the plot directory also receives a reassignment map figure when a reassignment file is used. |
| Movie | `<dataset>.mp4` or `.gif`, dumped frame PNGs | The movie file goes to the selected folder; dumped frames are disposable and live under `.cache/vmi-analysis/movie_frames/`. |

Files under `.cache/vmi-analysis/` are disposable GUI artifacts (previews, frame dumps, thumbnails) and can be deleted at any time. The checkpoint, the bin histograms and the PES outputs are the durable results; back them up rather than the cache. The repository ignores `.nicegui/`, `.cache/` and analysis outputs, so they never appear in version control.

# 7 Energy calibration

The raw VMI radius is converted to energy with

$$E = \alpha r^2, \tag{1}$$

where $r$ is measured in *histogram* pixels and $\alpha$ is in electronvolts per pixel$^2$. Until an alpha is supplied, the spectrum axis is the squared radius in pixel$^2$. The intensity is never rescaled: the y axis is always *Intensity (arb. units)*, and alpha changes only the x axis.

## 7.1 The Xe 4d reference

The gas-phase xenon 4d doublet has ionisation energies

$$\mathrm{IP}(4d_{3/2}) = 69.537 \text{ eV}, \qquad \mathrm{IP}(4d_{5/2}) = 67.548 \text{ eV}. \tag{2}$$

The two peaks appear at squared radii $q_{3/2}$ and $q_{5/2}$ in pixel$^2$, and the photon energy $h\nu$ is common to both:

$$\alpha q_{3/2} = h\nu - \mathrm{IP}(4d_{3/2}), \qquad \alpha q_{5/2} = h\nu - \mathrm{IP}(4d_{5/2}). \tag{3}$$

Solving the pair gives both the calibration constant and the photon energy:

$$\alpha = \frac{\mathrm{IP}(4d_{3/2}) - \mathrm{IP}(4d_{5/2})}{q_{5/2} - q_{3/2}}, \qquad h\nu = \mathrm{IP}(4d_{3/2}) + \alpha q_{3/2}. \tag{4}$$

The higher-$q$ peak belongs to the $4d_{5/2}$ line, because the smaller ionisation energy leaves more kinetic energy and therefore a larger radius.

### 7.2 Procedure

1. Run Centroiding with *Abel inversion* checked.
2. Tick *Fit Xe 4d doublet*. Optionally set *Xe fit min* and *Xe fit max* in pixel$^2$ to isolate the doublet from other lines; without limits the fit uses the two most prominent peaks.
3. Run. The console prints the fitted peak positions, the implied alpha, and the implied photon energy. The report is advisory: it never changes the calibration and never alters the saved figures.
4. Copy the reported alpha into the $\alpha$ field and run again (the localisation checkpoint is reused, so this is fast).
5. If desired, set the spectrum axis limits, remembering that they are in the displayed units: pixel$^2$ without alpha, eV with alpha.

If the fit finds fewer than two resolved peaks in the window, or too few samples, the console explains why and no calibration is reported. The reported uncertainties are formal fit errors and do not include systematic effects. Alpha is defined in histogram pixels, so it must be recalibrated if `Histogram resolution factor` changes.

### 7.3 Working with the spectrum file

`energy_spectrum.csv` has the columns `r_px`, `r2_px2`, `energy`, `dN_per_dr` and `intensity`. The first three are geometric, the fourth is the density per radius bin, and the fifth is the unchanged intensity. Only `energy` depends on alpha; the last column keeps its historical name from the point of view of the detector. Plot `energy` against `intensity` for the calibrated spectrum.

## 8 Command-line reference

Every stage is a standalone program. Run it from the repository root with `uv run python -m <module> --help` to see all options, or with the virtual-environment interpreter directly. The interface only adds a web form, job control, and result display; it never passes a configuration file.

| Module | Purpose and main inputs |
|---|---|
| `vmi_analysis.gmd.build_gmd_map` | Build the frame-to-GMD map; `--h5-dir`, `--gmd-path`, `--output-path`, `--n-files`. |
| `vmi_analysis.imaging.h5_play_frames_movie` | Render or stream a movie; `--h5-dir`, `--n-frames`, `--cmap`, `--log-scale`, `--headless`, `--dump-frames`. |
| `vmi_analysis.imaging.h5_accumulate_fast_gmd` | Fast accumulation; `--h5-dir`, `--output-path`, `--im-threshold`, `--simple-sum`, `--enable-gmd`, `--no-show`. |
| `vmi_analysis.centroiding.compute_dark` | Mean or median dark frame; `--dark-dir`, `--method`, `--frames`, `--output-dir`. |
| `vmi_analysis.centroiding.process_andor_vmi_PICASSO_v5_h5_batch` | Picasso localisation and histogram; `--h5-dir`, `--output-dir`, `--box-size`, `--min-net-gradient`, `--fitting-method`, `--alpha`, `--fit-xe-4d`. |
| `vmi_analysis.centroiding.vmi_histogram_gmd_bins` | GMD binning; `--checkpoint-path`, `--gmd-map-path`, `--output-dir`, `--thresholds`, `--weight-mode`, `--weight-power`. |
| `vmi_analysis.circularization.circularization` | Distortion correction of one image; `--input`, `--output`, `--shape`, `--center`, `--extracted`, `--reassign-file`, `--no-jacobian`. |
| `vmi_analysis.circularization.process_gmd_bins` | PES chain and Abel inversion; `--weight-suffix`, `--bin-dir`, `--output-dir`, `--inversion-backend`, `--pyabel-method`, `--use-cache`, `--no-sho` |

### 8.1 Conventions shared by all programs

- Path options marked required have no defaults: point them at your own data. Relative paths resolve from the program's working directory.
- `--config PATH` is optional and never loaded implicitly; explicit command-line values override the file. The interface never uses it.
- `--no-show` suppresses interactive Matplotlib windows, which is how GUI jobs run. Set `MPLBACKEND=Agg` for headless batch use.
- All programs print progress and a summary to the console; redirect it to a file for long runs.

### 8.2 Environment variables

| Variable | Default | Effect |
|---|---|---|
| `VMI_GUI_PORT` | 3157 | Port for the web interface. |
| `VMI_GUI_NO_SHOW` | unset | Set to 1 to suppress the browser window. |
| `VMI_WORKSPACE_DIR` | checkout root | Workspace used for saved settings and relative default paths, important for installed copies. |
| `NICEGUI_STORAGE_PATH` | `.nicegui` | Overrides where saved settings are stored. |
| `CPBASEX_DIR` | bundled vendor copy | Root of the MATLAB CPBASEX checkout used by the MATLAB backend. |
| *MPLBACKEND* | TkAgg in one script | Forces a non-interactive Matplotlib backend, for example `Agg`. |

## 9 Troubleshooting and constraints

**The server exits immediately.** The port is in use. Read the printed suggestion, set `VMI_GUI_PORT` to a free port, and rerun. Ports in the Windows excluded ranges are reported as unavailable.

**The server stops when I close a tab.** This is intended: the server shuts down after the last client disconnects. Keep one tab open while jobs run, or start the server again.

**A job waits forever.** Look at the waiting reason: it names the conflicting job or path. Another job writes an overlapping file, or the panel's previous job is still active. Wait, or stop the conflicting job in its own panel.

**Changing parameters does nothing.** Some stages resume from existing outputs. Accumulate skips the computation when the output text file exists; Centroiding skips localisation when the checkpoint exists; the Pipeline skips stages whose outputs exist. Delete the output (or untick the checkpoint in Clear outputs) to force a recomputation.

**GMD weighting has no effect.** The map may be missing, partial, or all weights are 1.0. A map built with *First N files* only covers those files; frames outside it silently keep weight 1.0.

**MATLAB/PES fails.** The MATLAB backend needs MATLAB and the basis file `G_r512_k512_l10.h5`. Generate it once with `save_gData.m`; expect tens of minutes. Use the PyAbel backend to avoid MATLAB entirely. MATLAB bins are processed sequentially by design.

**PES reports missing calibration.** Both files in `circularization/` are required and are not generated by this repository. Supply them before running.

**The frame histogram or movie disagrees with another tool.** Frame enumeration differs between tools: GMD Map and Dark Frame use `RAW-*.h5`, while Movie, Accumulate and the frame histogram use all `*.h5` files sorted by name. Extra HDF5 files change indices.

**Out-of-memory during dark computation.** The median method holds the selected dark frames in memory. Reduce *Dark frame count* or use the mean method.

**Some GMD statistics look wrong.** Frames with zeroed metadata carry a timestamp of 0 and match the first GMD entry. Exclude them when judging matching quality.

**Outputs overwrite silently.** The file picker does not prompt on save, and analysis outputs are overwritten. Choose a new output directory or file name when preserving an earlier result.

**I cannot reopen old results.** Only some panels offer *View saved result(s)*: GMD Map, Movie, Dark Frame, Accumulate and Centroiding. Circularize, PES, Binning and Pipeline have no such button; rerun or open the files with another tool.

# A Glossary

**VMI** Velocity-map imaging: an electron-optics configuration that maps photoelectron velocity to a position on the detector. The measured image is a 2D projection of the 3D momentum distribution.

**GMD** Gas monitor detector: records the pulse energy of each free-electron-laser shot, used to weight or bin frames.

**FEL** Free-electron laser, the photon source; it drives the camera at about 25 Hz together with the acquisition system.

**Centroiding** Estimating the sub-pixel position and intensity of each electron spot instead of counting whole pixels.

**Picasso** The localisation library used for spot detection and fitting.

**Abel inversion** Reconstruction of a 3D distribution from its 2D projection, assuming cylindrical symmetry about the detector axis.

$\beta$ **parameters** Coefficients of the angular distribution of photoelectrons; $\beta_2$ is the leading anisotropy term.

$\alpha$ Energy calibration constant, electronvolts per squared histogram pixel.

**Checkpoint** The `vmi_results_raw.h5` file of localised events, used both as an archive and as a resume token.

**GMD bin** A group of frames with similar GMD values; each bin gets its own 2D histogram and its own PES result.

**Weight suffix** The file-name suffix that distinguishes unweighted, GMD-weighted and normalised bin files.

**Workspace** The directory that holds saved settings and relative default paths; the repository root in a checkout.

# B Formulas used by the pipeline

**Energy calibration.** $E = \alpha r^2$, with $r$ in histogram pixels and $\alpha$ in eV/pixel$^2$; the xenon relations in Section 7 determine $\alpha$ and $h\nu$.

**Angular distribution.** For one-photon ionisation with cylindrical symmetry, the photoelectron angular distribution is

$$I(\theta) \propto 1 + \beta_1 P_1(\cos\theta) + \beta_2 P_2(\cos\theta) + \beta_3 P_3(\cos\theta) + \beta_4 P_4(\cos\theta), \tag{5}$$

where $P_\ell$ are Legendre polynomials and $\theta$ is measured from the light polarisation axis. The pipeline reports $\beta_2$ to $\beta_{10}$; for the usual experimental geometry the odd terms vanish.

**Abel inversion and the spectrum.** The 2D image is the line-of-sight integral of the 3D distribution. Abel inversion reconstructs the central slice, and the radial integral of that slice is the speed distribution, converted to an energy spectrum with the calibration above. The pipeline centres the image, applies the distortion correction and an edge taper before inversion so that the reconstruction is not dominated by the detector hole or the image boundary.

**GMD weighting.** The per-frame weight is

$$w_f = \frac{\overline{\mathrm{GMD}}}{\mathrm{GMD}_f}, \tag{6}$$

optionally raised to a power. It corrects the yield of a frame for its pulse energy; it cannot change the shape of a single-frame distribution.

**GMD binning.** A frame belongs to bin $b$ when $t_{b-1} \leq \mathrm{GMD} < t_b$, with the first and last bins open-ended.